Vulnerability assessment
OpenVAS/Greenbone

Fagnani Marco, Radice Lorenzo

May 19, 2026

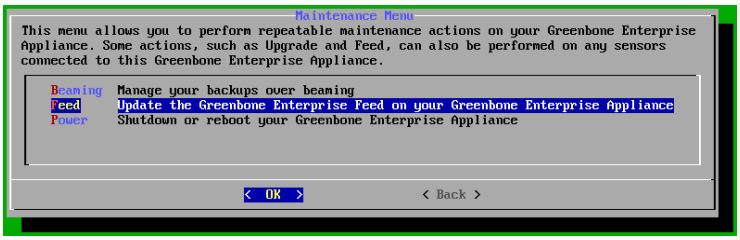
Contents

Chapter 1

Initial configuration

1.1 Greenbone

Greenbone/OpenVAS is an open-source vulnerability scanning tool that provides comprehensive security assessments for IT systems. To set up Greenbone/OpenVAS, firstly download the official Greenbone/OpenVAS OVA file from the Greenbone website.
https://www.greenbone.net/en/openvas-free/
Then, import the OVA file into VirtualBox 6.1 or higher. Assign at least 5GB of RAM and 2 CPU cores to the virtual machine. It's better to use about 12GB of RAM. For the first configuration, use the bridge network adapter, which allows the virtual machine to be on the same network as the host machine. Start the Greenbone/OpenVAS virtual machine and follow the on-screen instructions to complete the initial setup. Assign a user for the web interface. Once the setup is complete, access the Greenbone/OpenVAS update the feed by going to `Maintenance>Feed>Update` and wait it to synchronize (ususally hours).



1.2 Debian

Debian is an open-source operating system based on GNU/Linux. It will be used as a base to create a vulnerable machine. We used the latest stable version of Debian, which is Debian 13 Trixie.

1.2.1 Apache

Install Apache web server and create weak certificates for it. configure Apache to allow the use of weak protocols and ciphers, such as SSLv3, TLS1.1, DES, RC4 and MD5.

```
#!/bin/bash

apt-get update
apt-get install -y apache2 php libapache2-mod-php
rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*

a2enmod ssl
a2enmod rewrite
a2enmod headers
a2enmod proxy
a2enmod proxy_http
a2dissite 000-default.conf
echo "ServerName 127.0.0.1" | tee /etc/apache2/conf-available/servername.conf
a2enconf servername

output_dir=certs
mkdir -p $output_dir
cd $output_dir
openssl genrsa -out weak.key 1024
openssl req -new -key weak.key -out weak.csr -subj
↪  "/C=XX/ST=Lab/L=Vulnbox/O=Insecure/OU=Testing/CN=localhost"
openssl x509 -req -days 1 -in weak.csr -signkey weak.key -out weak.crt -sha1
#openssl x509 -req -days 3650 -in weak.csr -signkey weak.key -out weak_md5.crt
↪  -md5

mkdir -p /etc/apache2/ssl
cp weak.* /etc/apache2/ssl/

tee /etc/apache2/sites-available/vulnbox.conf  << EOF
<VirtualHost *:80>
    ServerName localhost

    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:8000/
    ProxyPassReverse / http://127.0.0.1:8000/

    # No security headers
</VirtualHost>

<VirtualHost *:443>
    ServerName localhost

    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/weak.crt
    SSLCertificateKeyFile /etc/apache2/ssl/weak.key

    # Allow TLS 1.0 only if OpenSSL supports it
    SSLProtocol all -SSLv3 -TLSv1.3 -TLSv1.2 -TLSv1.1
    #SSLProtocol all -SSLv3

    SSLCipherSuite ALL:NULL:EXPORT:LOW:MEDIUM:DES:RC4:MD5:@SECLEVEL=0
    #SSLCipherSuite aNULL:EXPORT:LOW:ALL:@SECLEVEL=0
    #SSLCipherSuite HIGH:!aNULL:@SECLEVEL=0
```

```
    #SSLCipherSuite ALL:NULL:EXPORT:LOW:@SECLEVEL=0

    SSLHonorCipherOrder off

    #SSLCompression on
    SSLSessionTickets On

    SSLOpenSSLConfCmd MinProtocol SSLv3
    SSLOpenSSLConfCmd CipherString DEFAULT:@SECLEVEL=0

    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:8000/
    ProxyPassReverse / http://127.0.0.1:8000/

    Header always unset Strict-Transport-Security
    Header always unset X-Frame-Options
    Header always unset X-Content-Type-Options
</VirtualHost>
EOF

a2ensite vulnbox.conf

tee /etc/php/7.4/apache2/php.ini << EOF
display_errors = On
display_startup_errors = On
allow_url_fopen = On
allow_url_include = On
disable_functions =
open_basedir =
expose_php = On
EOF

systemctl restart apache2
```

Any website can be used as exposed service. In our case we used an old project of ours, properly modified to be vulnerable.
`git clone -b vulnerable https://github.com/ozneroL541/artigianato-online.git`
To run it execute the following commands in the terminal:

```
cd artigianato-online/src
docker compose up --remove-orphans -d
```

1.2.2 Cups

Install the CUPS printing system and configure it to allow remote administration without authentication.

```
#!/bin/bash

apt-get update
apt-get install -y cups
rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*

tee /etc/cups/cupsd.conf << EOF
Listen 0.0.0.0:631

<Location />
```

```
  Order allow,deny
  Allow all
</Location>

<Location /admin>
  Order allow,deny
  Allow all
</Location>

WebInterface Yes
DefaultAuthType Basic
EOF

systemctl enable --now cups
```

1.2.3 Docker Vulnet

To simulate a complex network environment, we are using a docker container with multiple vulnerable services. The scope of this network is to show the capabilities of Greenbone/OpenVAS in scanning a complex system. To set up the network we used the following docker compose file:

```
################################################################
#  Vulnerability Assessment Lab
#
#  Network: 172.30.0.0/22
#    host     - 172.30.0.1/24
#    vulnbox1 - 172.30.0.2/24 172.30.1.1/24
#    vulnbox2 - 172.30.1.2/24 172.30.2.1/24
#    vulnbox3 - 172.30.2.2/24
#    vulnbox4 - 172.30.2.3/24
#    vulnbox5 - 172.30.2.4/24 172.30.3.1/24
#    vulnbox6 - 172.30.3.2/24
################################################################

networks:
  # Bridge { directly reachable from host
  net0:
    driver: bridge
    driver_opts:
      com.docker.network.bridge.name: "vulnlab0"
    ipam:
      config:
        - subnet: 172.30.0.0/24
          gateway: 172.30.0.1

  # Internal macvlan segments { isolated from host, routed only through vulnbox1
  net1:
    driver: bridge
    driver_opts:
      com.docker.network.bridge.name: "vulnlab1"
    internal: true          # blocks direct host - net1 traffic
    ipam:
      config:
        - subnet: 172.30.1.0/24
          gateway: 172.30.1.254

```

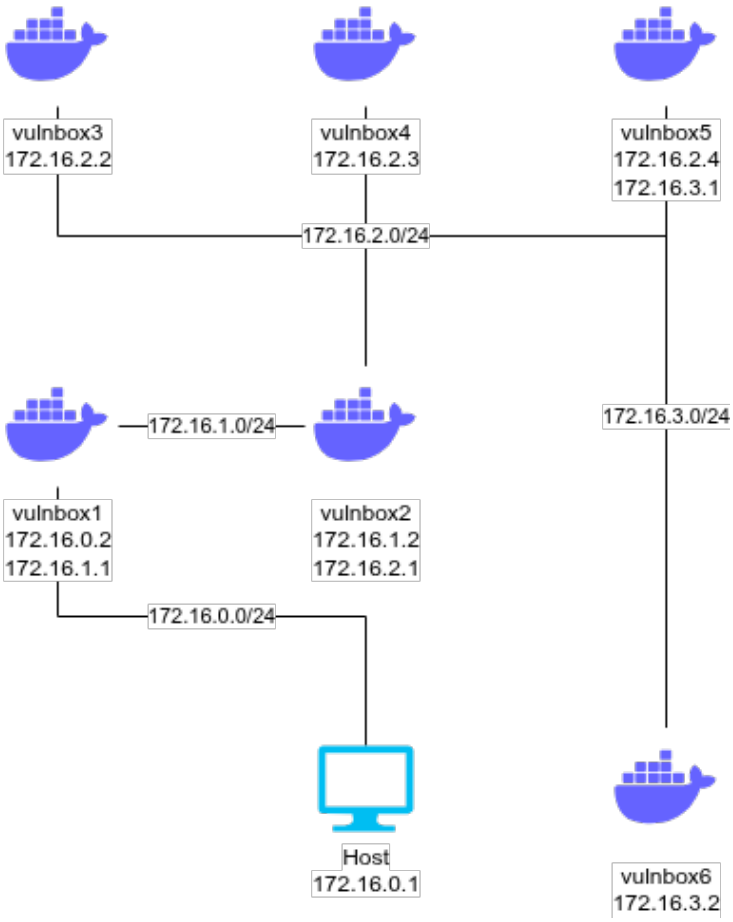


Figure 1.1: Docker network configuration

```
36    net2:
37      driver: bridge
38      driver_opts:
39        com.docker.network.bridge.name: "vulnlab2"
40      internal: true          # blocks direct host - net2 traffic
41      ipam:
42        config:
43          - subnet: 172.30.2.0/24
44            gateway: 172.30.2.254
45
46    net3:
47      driver: bridge
48      driver_opts:
49        com.docker.network.bridge.name: "vulnlab3"
50      internal: true
51      ipam:
```

```
      config:
        - subnet: 172.30.3.0/24
          gateway: 172.30.3.254


services:
  # --- VULNBOX 1 { Entering node -------------------------
  vulnbox1:
    build:
      context: ./network/vulnbox-1
      dockerfile: Dockerfile
    image: vulnbox1:latest
    container_name: vulnbox1
    hostname: vulnbox1
    networks:
      net0:
        ipv4_address: 172.30.0.2
      net1:
        ipv4_address: 172.30.1.1
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 2 { Apache Struts 2.3.31 (CVE-2017-5638) ----
  vulnbox2:
    build:
      context: ./network/vulnbox-2
      dockerfile: Dockerfile
    image: vulnbox2:latest
    container_name: vulnbox2
    hostname: vulnbox2
    networks:
      net1:
        ipv4_address: 172.30.1.2
      net2:
        ipv4_address: 172.30.2.1
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 3 { DVWA -----------------------------------
  vulnbox3:
    build:
      context: ./network/vulnbox-3
      dockerfile: Dockerfile
    image: vulnbox3:latest
    container_name: vulnbox3
    hostname: vulnbox3
    networks:
      net2:
        ipv4_address: 172.30.2.2
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 4 { SSH weak credentials -------------------
```

```
  vulnbox4:
    build:
      context: ./network/vulnbox-4
      dockerfile: Dockerfile
    image: vulnbox4:latest
    container_name: vulnbox4
    hostname: vulnbox4
    networks:
      net2:
        ipv4_address: 172.30.2.3
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 5 { Samba ----------------------------------
  vulnbox5:
    build:
      context: ./network/vulnbox-5
      dockerfile: Dockerfile
    image: vulnbox5:latest
    container_name: vulnbox5
    hostname: vulnbox5
    networks:
      net2:
        ipv4_address: 172.30.2.4
      net3:
        ipv4_address: 172.30.3.1
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 6 { MySQL no root password -----------------
  vulnbox6:
    build:
      context: ./network/vulnbox-6
      dockerfile: Dockerfile
    image: vulnbox6:latest
    container_name: vulnbox6
    hostname: vulnbox6
    networks:
      net3:
        ipv4_address: 172.30.3.2
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped
```

To ensure proper network setup it was crucial to set the correct routes throgh all the containers, so that they could communicate with each other in the way we wanted. To do so we wrote the following script:

```
#!/bin/bash
# Stop and remove any existing containers
docker compose down --remove-orphans
# Start the vulnbox network
docker compose up --remove-orphans -d
# Allow forwarding between containers
```

```
iptables -I FORWARD 1 -s 172.30.0.0/22 -d 172.30.0.0/22 -j ACCEPT
# Remove the default IP addresses from the bridge interfaces
ip addr del 172.30.1.254/24 dev vulnlab1
ip addr del 172.30.2.254/24 dev vulnlab2
ip addr del 172.30.3.254/24 dev vulnlab3
# Host
# Tell your physical machine to send lab traffic to vulnbox1
ip route replace 172.30.1.0/24 via 172.30.0.2
ip route replace 172.30.2.0/24 via 172.30.0.2
ip route replace 172.30.3.0/24 via 172.30.0.2
# vulnbox1
docker exec -u 0 vulnbox1 sysctl -w net.ipv4.ip_forward=1
docker exec -u 0 vulnbox1 ip route replace 172.30.2.0/24 via 172.30.1.2
docker exec -u 0 vulnbox1 ip route replace 172.30.3.0/24 via 172.30.1.2
docker exec -u 0 vulnbox1 ip route add default via 172.30.0.1
# vulnbox2
docker exec -u 0 vulnbox2 sysctl -w net.ipv4.ip_forward=1
docker exec -u 0 vulnbox2 ip route replace 172.30.3.0/24 via 172.30.2.4
docker exec -u 0 vulnbox2 ip route add default via 172.30.1.1
# vulnbox3
docker exec -u 0 vulnbox3 ip route add 172.30.3.0/24 via 172.30.2.4
docker exec -u 0 vulnbox3 ip route add default via 172.30.2.1
# vulnbox4
docker exec -u 0 vulnbox4 ip route replace 172.30.3.0/24 via 172.30.2.4
docker exec -u 0 vulnbox4 ip route add default via 172.30.2.1
# vulnbox5
docker exec -u 0 vulnbox5 sysctl -w net.ipv4.ip_forward=1
docker exec -u 0 vulnbox5 ip route add default via 172.30.2.1
# vulnbox6
docker exec -u 0 vulnbox6 ip route add default via 172.30.3.1
```

To let Greenbone/OpenVAS scan the containers, we had to crete further rules. Since the VM is inteded as a vulnerable machine, we had no intention to let the firewall block any traffic, therefore, firstly, we flushed all its rules and then we build up our own ruleset, which allows all the traffic from the Greenbone/OpenVAS to the container's network.

```
#!/bin/bash

# DO NOT RUN THIS ON YOUR HOST MACHINE
# VM ONLY

docker_interface="vulnlab0"
docker_subnet="172.30.0.0/22"
interface=$(ip -o -f inet addr show | awk '/scope global/ {print $2}' | head -n
↪  1)

./setup.sh
# Flush everything and start clean
iptables -F FORWARD
iptables -t nat -F POSTROUTING
nft flush chain ip raw PREROUTING

# Allow forwarding between containers
iptables -I FORWARD 1 -s $docker_subnet -d $docker_subnet -j ACCEPT

# Re-add Docker's default NAT rules
iptables -t nat -A POSTROUTING -s $docker_subnet ! -o $docker_interface -j
↪  MASQUERADE
```

```
iptables -t nat -A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE

# Allow forwarding between eth and bridge
iptables -I FORWARD 1 -i $interface -o $docker_interface -j ACCEPT
iptables -I FORWARD 1 -i $docker_interface -o $interface -j ACCEPT

# Allow Docker chain
iptables -I DOCKER 1 -i $interface -d $docker_subnet -j ACCEPT
```

vulnbox1

The first vulnbox was intentended to be vulnerable to log4Shell (CVE-2021-44228), a critical vulnerability in the Apache Log4j library. However, since the container was handmade, Greenbone/OpenVAS was not able to detect it.

```
# Vulnbox 1 - Log4Shell
# Pivot node for networking
FROM maven:3.9.9-eclipse-temurin-8 AS build

WORKDIR /build
COPY pom.xml .
RUN mvn -q -e -DskipTests
↪  org.apache.maven.plugins:maven-dependency-plugin:3.3.0:go-offline

COPY src ./src
RUN mvn -q -DskipTests package

# ---- runtime ----
FROM eclipse-temurin:8-jre-jammy

# install tools for your lab
RUN apt-get update -o Acquire::AllowInsecureRepositories=true \
    -o Acquire::AllowDowngradeToInsecureRepositories=true && \
    apt-get install -y --allow-unauthenticated iproute2 curl && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*

WORKDIR /app
COPY --from=build /build/target/vulnapp-1.0-jar-with-dependencies.jar .

EXPOSE 8080

CMD ["java", "-jar", "vulnapp-1.0-jar-with-dependencies.jar"]
```

Even it the vulnerability was not detected, the main role of the first vulnbox was to route all the traffic to the docker network and vice versa and, in that sense, it was successful.

vulnbox2

The second vulnbox present a vulnerable version of the Apache Struts framework, which is affected by the CVE-2017-5638 vulnerability.

```
# Apache Struts 2.3.31 { CVE-2017-5638
# The Content-Type header is parsed by the Jakarta Multipart parser.
# A malformed value triggers OGNL expression evaluation → unauthenticated RCE.
```

```
# This is the exact vulnerability used in the 2017 Equifax breach.

FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

# Install Java 8 + Tomcat 8 + wget
RUN apt-get update -qq && \
    apt-get install -y -qq \
        openjdk-8-jdk-headless \
        wget \
        curl \
        unzip \
        iproute2 && \
    rm -rf /var/lib/apt/lists/*

ENV JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
ENV PATH="$JAVA_HOME/bin:$PATH"

# -- Tomcat 8.5 ------------------------------------------------
ENV TOMCAT_VERSION=8.5.87
ENV CATALINA_HOME=/opt/tomcat

RUN wget -q
↪  "https://archive.apache.org/dist/tomcat/tomcat-8/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.g
↪  \
        -O /tmp/tomcat.tar.gz && \
    mkdir -p "$CATALINA_HOME" && \
    tar xzf /tmp/tomcat.tar.gz -C "$CATALINA_HOME" --strip-components=1 && \
    rm /tmp/tomcat.tar.gz

# -- Struts 2.3.31 showcase WAR (contains the vulnerable REST plugin) --
ENV STRUTS_VERSION=2.3.31

RUN wget -q
↪  "https://archive.apache.org/dist/struts/${STRUTS_VERSION}/struts-${STRUTS_VERSION}-apps.zip"
↪  \
        -O /tmp/struts-apps.zip && \
    unzip -q /tmp/struts-apps.zip
    ↪  "struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
        -d /tmp/struts-extract/ && \
    cp "/tmp/struts-extract/struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
        "${CATALINA_HOME}/webapps/struts2-showcase.war" && \
    rm -rf /tmp/struts-apps.zip /tmp/struts-extract

# Remove default Tomcat apps to keep the attack surface clean/focused
RUN rm -rf \
    "$CATALINA_HOME/webapps/ROOT" \
    "$CATALINA_HOME/webapps/docs" \
    "$CATALINA_HOME/webapps/examples" \
    "$CATALINA_HOME/webapps/host-manager" \
    "$CATALINA_HOME/webapps/manager"

# Tomcat manager creds (useful for post-exploit exploration)
COPY tomcat-users.xml "$CATALINA_HOME/conf/tomcat-users.xml"

EXPOSE 8080


```

```
CMD ["sh", "-c", "$CATALINA_HOME/bin/catalina.sh run"]
```

vulnbox3

The third vulnbox is based on DVWA (Damn Vulnerable Web Application), a web application that is intentionally vulnerable to various types of attacks. We had no clue if Greenbone/OpenVAS would have been able to detect any vulnerabilities here, but since it's in fact the scope of such experiment, we decided to include it.

```
# Vulnbox 3 { DVWA
# Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web app designed for security training.
# It contains multiple vulnerabilities of varying difficulty, making it ideal for practice.

FROM vulnerables/web-dvwa

ENV MYSQL_PASS=p@ssw0rd
ENV HOSTNAME=vulnbox3


EXPOSE 80 443
```

vulnbox4

The fourth vulnbox is a simple Ubuntu 22.04 image with weak SSH credentials.

```
# Vulnbox 4 - SSH Service with Weak Credentials
# This container runs an SSH server with weak credentials for testing purposes.
FROM ubuntu:22.04

RUN apt-get update -qq && \
    apt-get install -y -qq openssh-server iproute2 && \
    rm -rf /var/lib/apt/lists/* && \
    useradd -m -s /bin/bash admin; \
    echo 'admin:admin123' | chpasswd; \
    useradd -m -s /bin/bash user; \
    echo 'user:password' | chpasswd; \
    mkdir -p /run/sshd && \
    sed -i 's/#PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config && \
    echo 'root:toor' | chpasswd

EXPOSE 22


CMD ["/usr/sbin/sshd", "-D"]
```

vulnbox5

The fifth vulnbox is a weak Samba server.

```
# Vulnbox 5 - Samba (EthernalBlue-era misconfiguration)
```

```
# This container runs a Samba server with a misconfiguration that allows for potential exploitation.

FROM dperson/samba

RUN mkdir -p /public /private && \
    chmod 777 /public && \
    chmod 700 /private

ENV USERID=1000 \
    GROUPID=1000


CMD ["-u", "guest;guest", \
     "-s", "public;/public;yes;no;yes;all;none", \
     "-s", "private;/private;no;no;no;all;none"]
```

vulnbox6

The sixth vulnbox is a MySQL server with no password.

```
FROM mysql:5.7

ENV MYSQL_ALLOW_EMPTY_PASSWORD=yes

RUN yum clean all && \
    yum install -y iproute && \
    yum clean all && \
    rm -rf /var/cache/yum
```

1.2.4 FTP

The FTP server is a simple instance of `vsftpd` with multiple weak configurations, such as:

```
local_enable=YES
write_enable=YES
pam_service_name=vsftpd
```

1.2.5 Samba

The Samba server is configured to allow insecure access to a shared folder.

```
#!/bin/bash

apt-get install -y samba

tee /etc/samba/smb.conf << EOF
[global]
   workgroup = WORKGROUP
   server string = Vuln Samba
   map to guest = Bad User
   guest account = nobody

   # Weak / legacy protocol support
   server min protocol = NT1
```

```
   ntlm auth = yes
   lanman auth = yes

   # Disable signing
   server signing = disabled

[vulnshare]
   path = /srv/samba/vuln
   browsable = yes
   writable = yes
   guest ok = yes
   read only = no
   force user = nobody
EOF

mkdir -p /srv/samba/vuln
chmod 777 /srv/samba/vuln
systemctl enable --now smbd
```

1.2.6 SSH

The SSH server is configured with weak configurations, such as:

```
PermitRootLogin yes
PasswordAuthentication no
```

1.2.7 Syslog

The Syslog server is configured to allow remote logging without authentication on port 514.

```
#!/bin/bash

#apt-get update
#apt-get install -y rsyslog
#apt-get clean
#rm -rf /var/lib/apt/lists/*

# Enable UDP + TCP syslog reception
cat << EOF > /etc/rsyslog.d/10-remote.conf
# Load modules
module(load="imudp")
input(type="imudp" port="514")

module(load="imtcp")
input(type="imtcp" port="514")

# Log EVERYTHING from remote hosts
*.* /var/log/remote.log

# No rate limiting (intentionally bad)
\$SystemLogRateLimitInterval 0
EOF

# Ensure rsyslog is enabled
systemctl enable rsyslog
systemctl restart rsyslog
```

```
27
28  mkdir -p /var/log/
29  chmod 777 -R /var/log/
```

1.3 Windows XP

Windows XP is an old operating system that is no longer supported by Microsoft, which makes it vulnerable to various types of attacks. Just it's mere presence in the network is a vulnerability, since it can be easily exploited by attackers.

To set up Windows XP, we just installed its virtual machine on VirtualBox and connected it to the same network of the other machines. To install all vulnerable services you should first acquire a Windows XP Pro Service Pack 1 iso, go to the Control Panel, Add/Remove Windows Components and we enabled every service that was enabled

Figure 1.2: Full laboratory's network

Then to enable the FTP server select **Internet Information Services (IIS)**, click **details** and you should enable it.

Then click ok and Windows will install all required packages, then reboot. The FTP service now should be installed. This version of Windows did not have

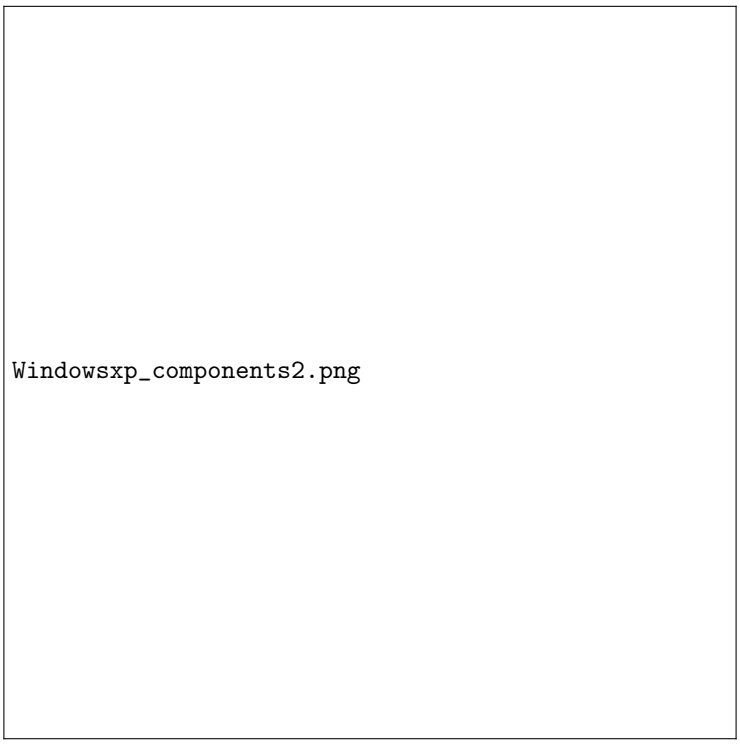


Figure 1.3: Full laboratory's network

a firewall enabled by default, and even so it was a primitive version so it should not pose issues.

1.4 Network configuration

The whole network configuration is an Host-only virtual network created with VirtualBox, which allows all the machines to communicate with each other and have no access to other networks, such as the Internet.

In order to create the virtual network and connect all the machines to it, we followed the following procedure:

1. Create a Host-only network in VirtualBox

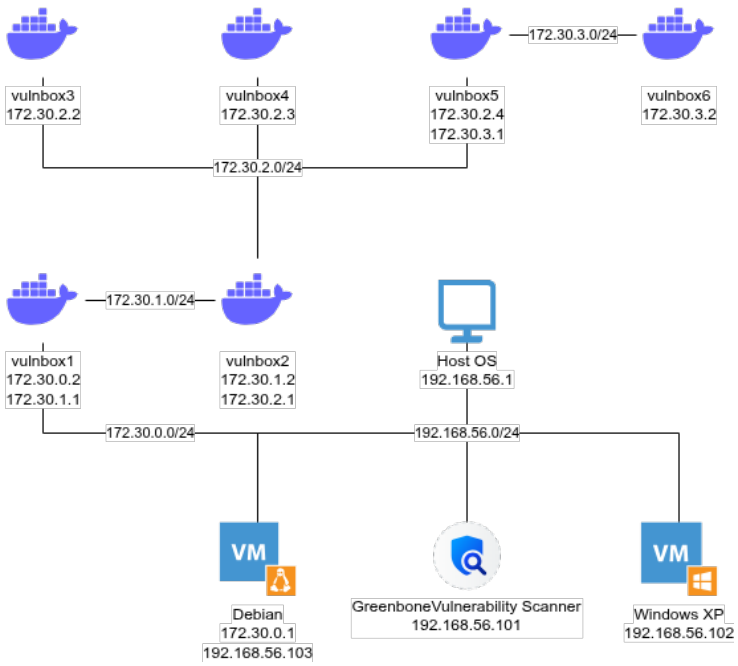
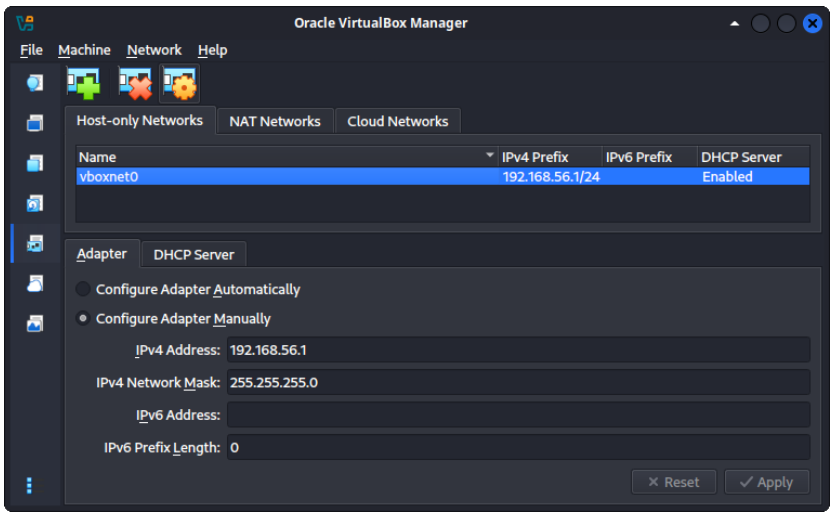
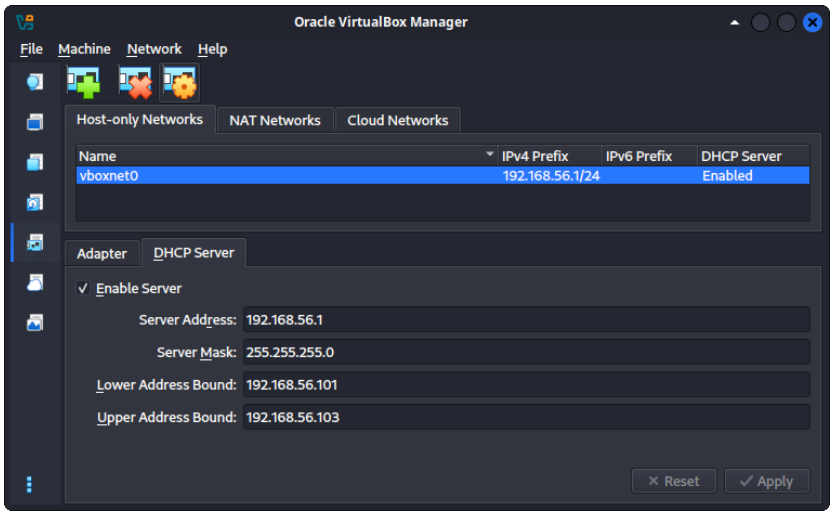


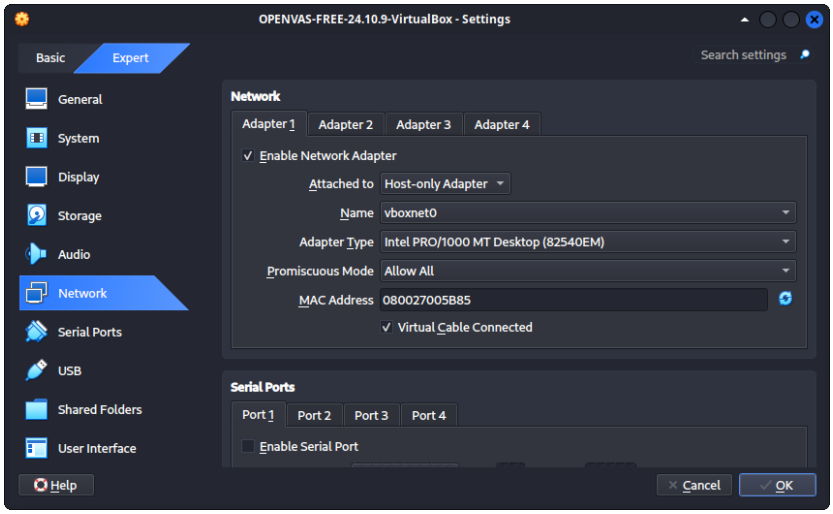
Figure 1.4: Full laboratory's network



2. Enable the DHCP server for the Host-only network enough to assign IP addresses to all the 3 machines



3. Go to settings of each machine and connect it to the Host-only network



Chapter 2

System Discovery

In this chapter we will describe how to perform a System Discovery using OpenVAS. There must be clarified that this type of scan is not a Vulnerability Scan, it's just a detection of the targets of a system, their open ports, the services running on them and its whole topology.

2.1 Get Debian VM IP address

Before performing the System Discovery, we need to know the IP address of the Debian VM, since we need to specify it as Greenbone default gateway to reach the docker network where the vulnboxes are located. Open the Debian VM, which credentials are:

- Username: root
- Password: root

Then go to the folder `/network-security/project/docker-vulnet/` and run the command `bash hardcore-setup.sh` to burn down the firewall and build up some rules which allow the Debian machine to redirect the traffic to the docker network.

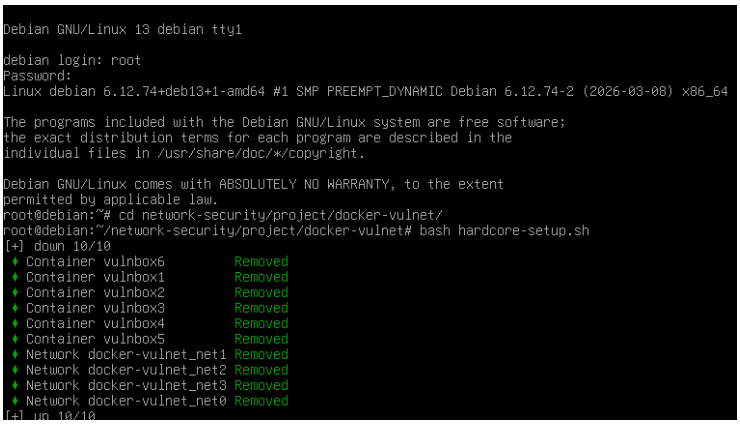


Figure 2.1: Debian VM - Initial setup

After that, run the command `ip addr | grep 192` to get the IP address of the Debian VM.

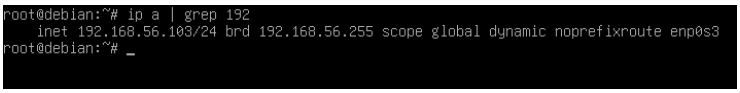


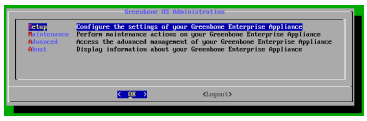
Figure 2.2: Debian VM - IP address

2.2 Configure Greenbone Gateway

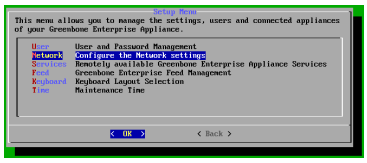
Before performing the System Discovery, we need to configure the Greenbone Gateway to reach the docker network where the vulnboxes are located. Open the OpenVAS VM, which credentials are:

- Username: admin
- Password: admin

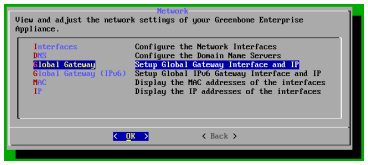
Go to **Setup¿Network¿Global Gateway** and change the default gateway according to the IP address of the Debian VM, which we got in the previous section. Then save the configuration.



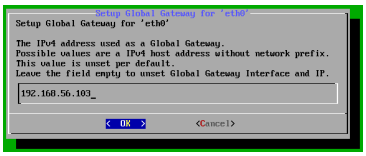
(a) Greenbone VM - Main Section



(b) Greenbone VM - Global Gateway configuration

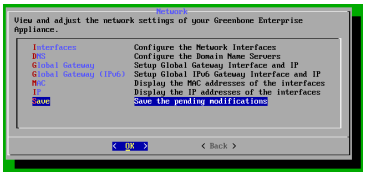


(c) Greenbone VM - Global Gateway configuration (2)



(d) Greenbone VM - Global Gateway configuration (3)

(e) Greenbone VM - Global Gateway configuration (3)



(f) Greenbone VM - Global Gateway configuration (3)

Figure 2.3: Greenbone VM - Gateway configuration

2.3 Greenbone Web Interface

Open the *About* section in the main menu of the Greenbone VM.

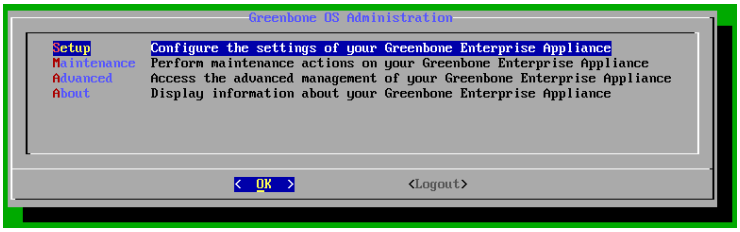


Figure 2.4: Greenbone VM - Main section

Check out the IP address of the VM.

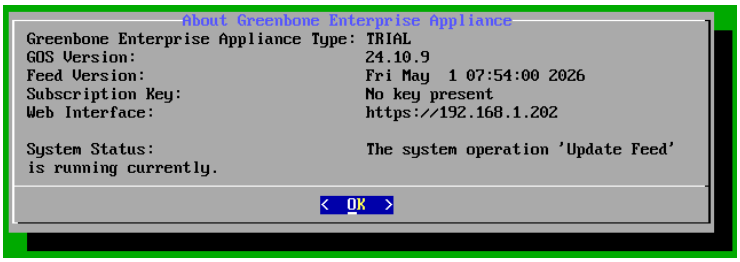


Figure 2.5: Greenbone VM - About section

Open a web browser and type the URL for the web interface in the search bar.

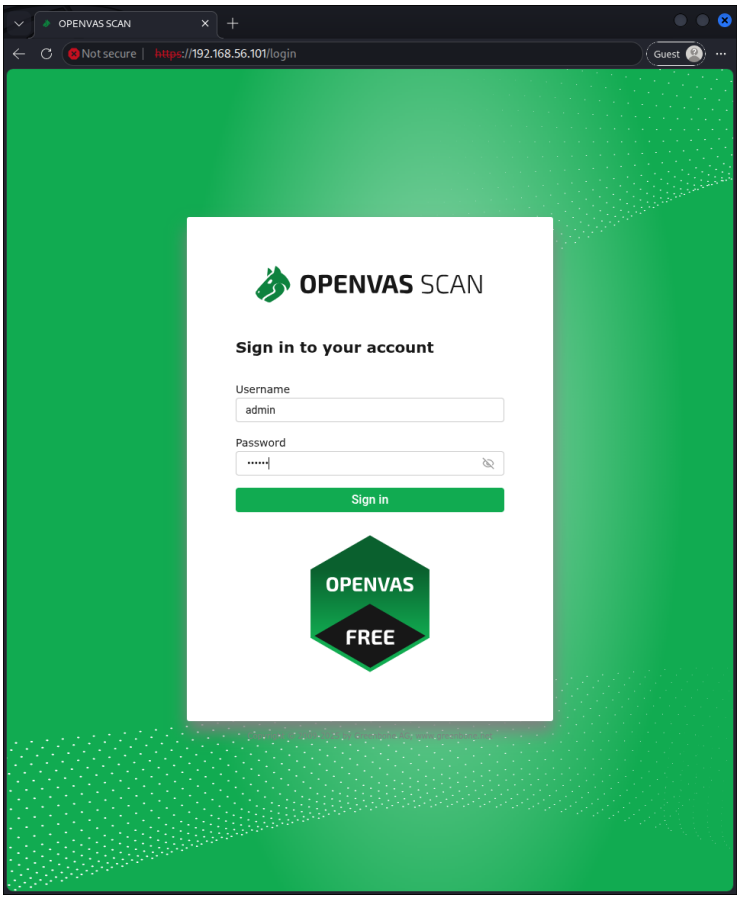


Figure 2.6: Greenbone Web Interface - Login page

Log in with the credentials:

- Username: admin
- Password: admin

You will see the main page of the web interface.

2.4 System Scan

Now go to the *Scans* section and click on *Tasks*.

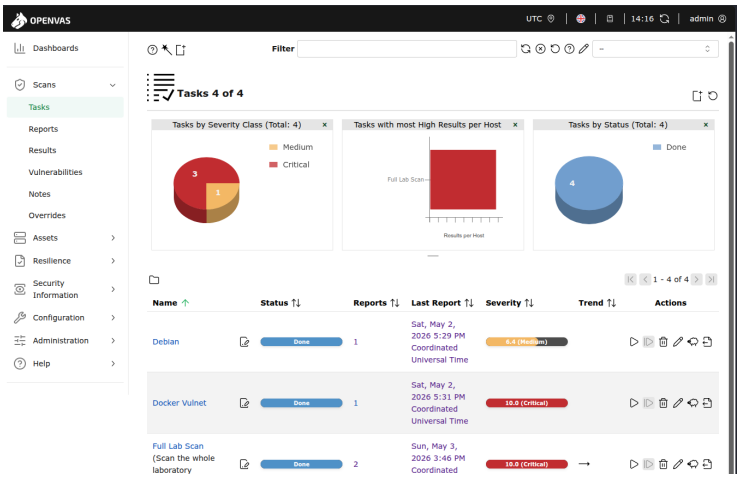


Figure 2.7: Greenbone Web Interface - Main page

Modify the *Docker Vulnet* task. Edit it by choosing *System Discovery* as the scan configuration, then save the task.

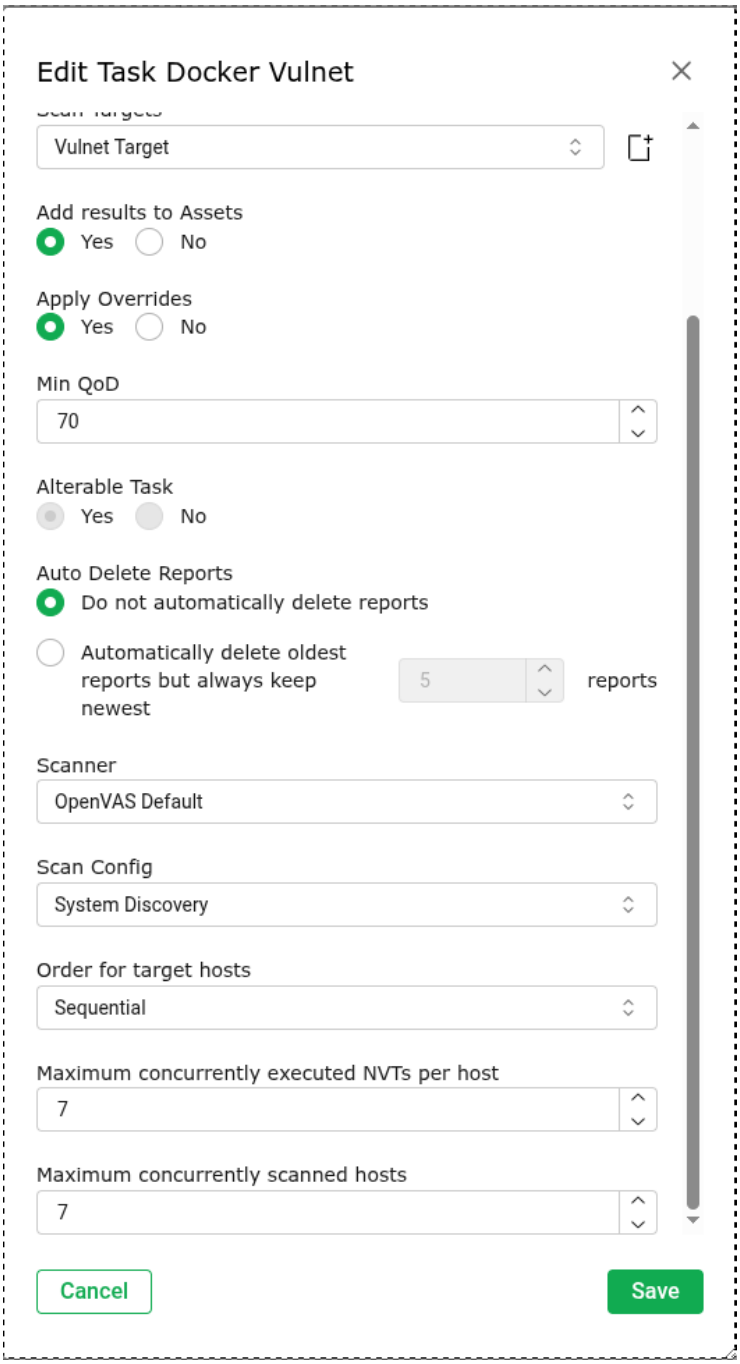


Figure 2.8: Greenbone Web Interface - Task modification

Now we can launch the task pressing the *Play* button.

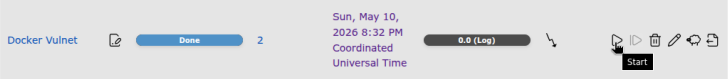


Figure 2.9: Greenbone Web Interface - Task launch

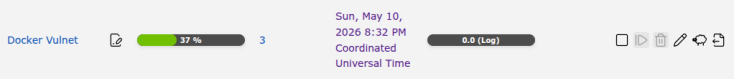


Figure 2.10: Greenbone Web Interface - Task progress

After about 10 minutes, the task will be completed and we can see the results. To see the results, click on the task and then on the *Report* tab. In the upper right corner, click on the *Remove filters* button to see all the results. This is required because by default the web interface shows only the results with a severity score, but since the *System Discovery* scan configuration doesn't detect vulnerabilities, all the results have a severity score of 0, so we need to remove the filters to see them all.

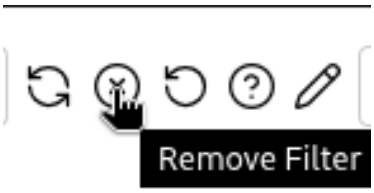


Figure 2.11: Greenbone Web Interface - Remove filters

Now we can see all the results of the scan which comprehends:

- The detected hosts in the given IP range
- The open ports on the hosts
- The services running on the open ports
- The topology of the system

Even if the scanned system only has 6 vulnboxes, as we can see, OpenVAS detected a lot more hosts, since in the OpenVAS view, each IP address corresponds to a host, and in the scanned system some vulnboxes have more than one IP address, therefore OpenVAS detected all of them as distinct hosts.

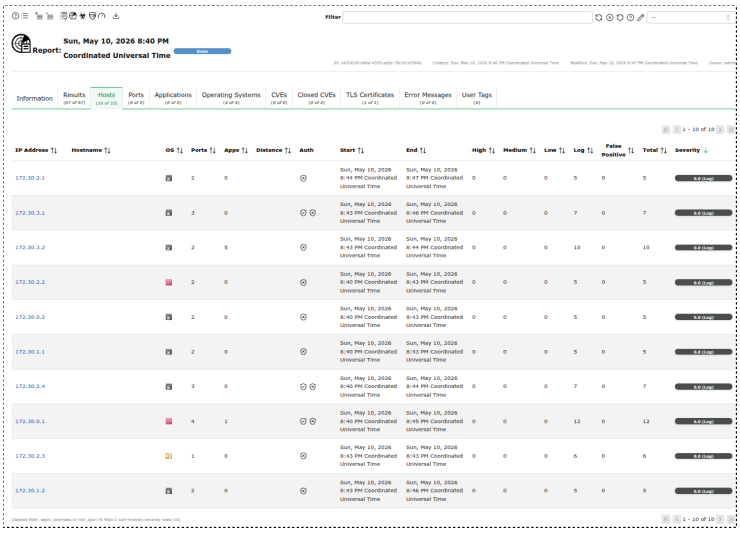


Figure 2.12: Greenbone Web Interface - Scan results

Operating System	CPE	Hosts	Severity
Linux Kernel	cpe:/o:linux:kernel	7	0.0 (Log)
Debian GNU/Linux 9	cpe:/o:debian:debian_linux:9	1	0.0 (Log)
Debian GNU/Linux 13	cpe:/o:debian:debian_linux:13	1	0.0 (Log)
Ubuntu 22.04	cpe:/o:canonical:ubuntu_linux:22.04	1	0.0 (Log)

Figure 2.13: Greenbone Web Interface - Operating Systems

The results of this scan can be also be seen in the *Assests¿Hosts* section of the web interface, where we can see all the detected hosts and their open ports and services.

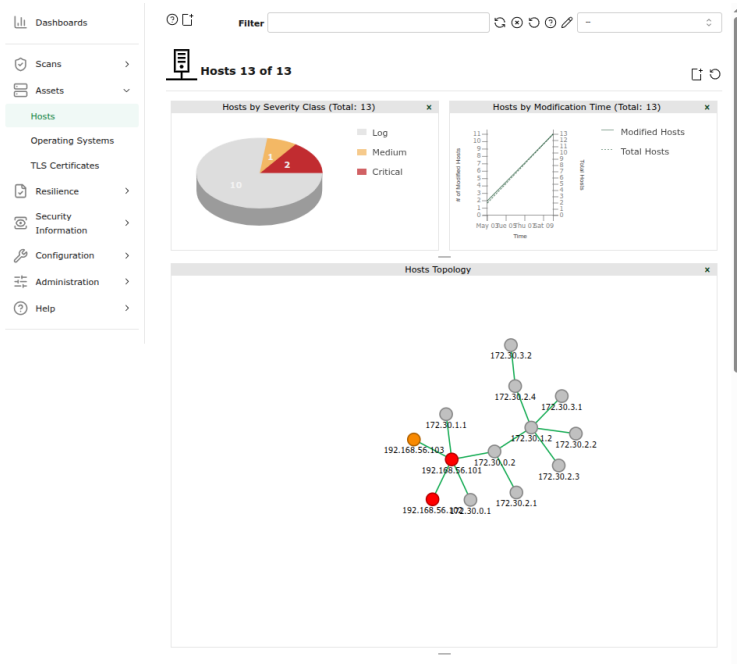
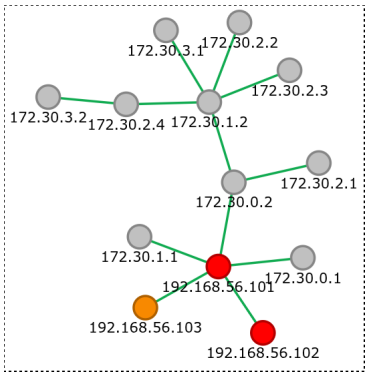
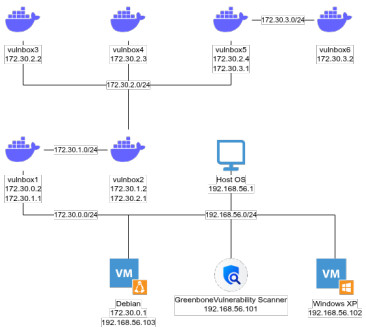


Figure 2.14: Greenbone Web Interface - Hosts section

We can here check if the detected topology of the system is correct, by comparing it with the one we designed.



(a) Greenbone Web Interface - Detected topology



(b) Designed topology

Figure 2.15: Greenbone Web Interface - Detected topology vs Designed topology

As we can see, the detected topology is correct, even if there are more hosts than vulnboxes, the overall topology is connected in the right way.

Chapter 3

OpenVAS Scan

In this chapter the most important aspects of OpenVAS are explained, with practical examples to give the reader an understanding of the tools usage.

3.1 OpenVAS Scan

In this section it will be explained how to start a scan (that could be just for an host, or a group of hosts). We will create a target, which means declaring which host (or group of hosts) to be scanned, which port range and protocol.

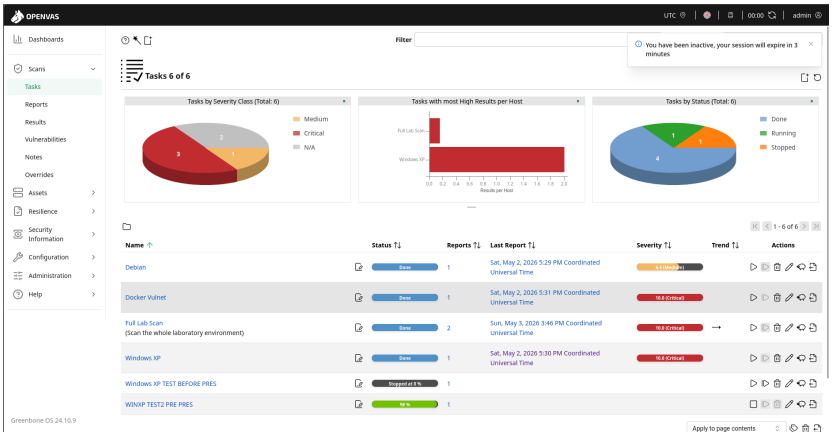


Figure 3.1: Scan windows

This is the scan windows, where you will create the tasks which are configurations for the targets and ports that are going to be scanned, how many NVTs (Network Vulnerability Test) are going to be used for the scan, how many concurrent hosts are being scanned at a given moment etc. We will get to all of the configuration in time.

3.1.1 Configuration → Ports

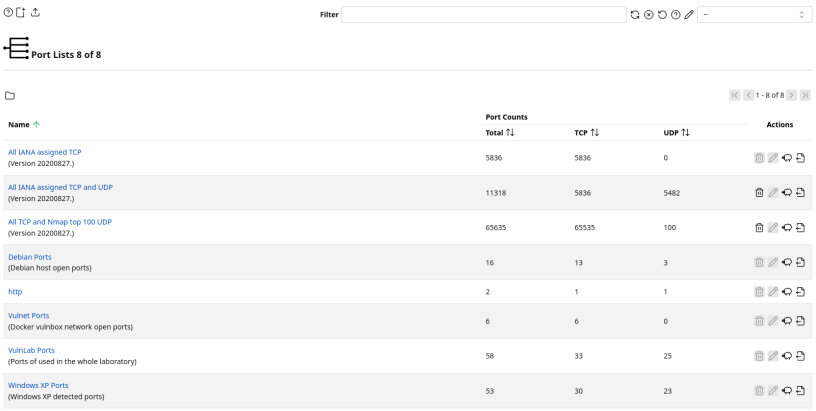


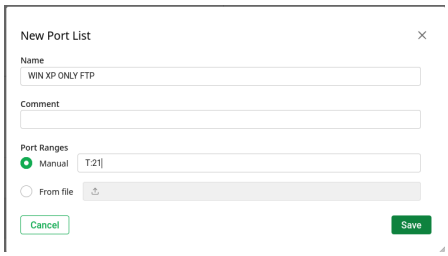
Figure 3.2: Port creation interface

This is the ports creation interface, you can create custom ports ranges looking for services that use TCP/UDP. For the most part OpenVAS works by giving us templates, in this case are for a complete scans of all IANA (Internet Assigned Numbers Authority) ports, which means it will scan for all 65536 ports only TCP, UDP, both etc. It really just depends on which preset is chosen.

OpenVAS encodes ports in a format like $T{:}1-1023,\ 4500{,}U{:}2-100,\ 3246$ which means that it will scan for TCP ports between 1 and 1023 and 4500 alone, and for UDP ports from 2 to 100 and 3246 alone. In this way you can define ports range and choose to do only TCP or UDP scans.

Under the hood OpenVAS uses Nmap to scan the ports, and just like it to scan UDP ports it will take considerably longer than TCP. This is because when a TCP request is made, the server either accept the connection or it rejects it, while for UDP being a connection-less protocol will wait until timeout. So beware about this aspect if you don't want the scan to take hours.

For our experiments we will create two presets: one for ftp service (port 21) and one for an http server (port 80). Go on the add port icon on the top of the interface (the little square with a plus), and you should get this popup.



(a) Port configuration limited to `TCP 21` (FTP).

(b) Port configuration limited to `TCP 21 and 80` (FTP).

Figure 3.3: Ports configuration

Those configurations are going to be used later to scan a Windows XP target with FTP and HTTP servers.

Credentials for target access

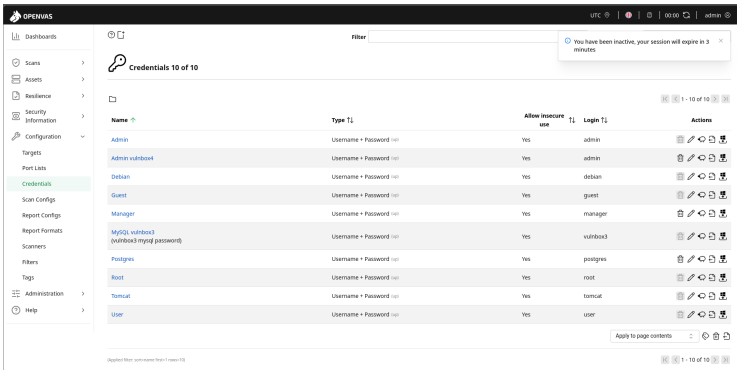
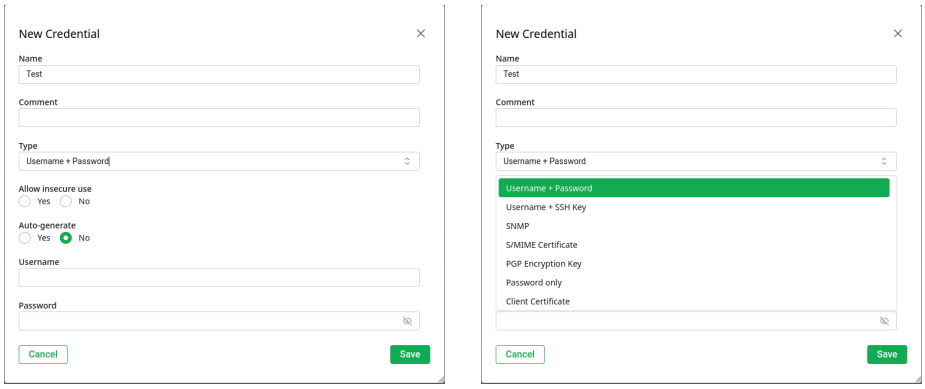


Figure 3.4: Credentials window

Credentials are used for deeper scans, you give the credentials to OpenVAS and it will try to login to the machine, scanning the host from within, looking at configuration files, installed packages etc. This is done to improve scan performance since it flags problems that are not visible from the outside, but may lead to exploitable vulnerabilities. This function can also be used to check for default credentials if they are in use or no.



(a) FTP Target Step 1 (b) FTP Target Step 2

Figure 3.5: Configuration for the FTP only target

As you can see many possibilities are available, setting both username and password, ssh access, certificates etc. But for our example this feature will not be used.

3.1.2 Target configuration

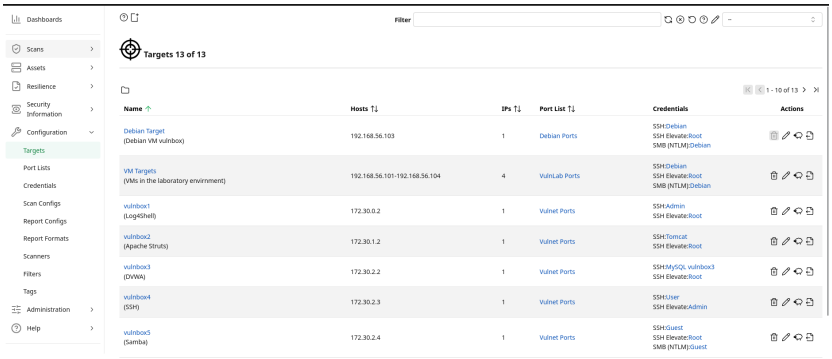


Figure 3.6: Target configuration interface

This is the target creation interface, which is used to create all targets that are going to be scanned expressed in single IP or a range of IPs (e.g. 192.168.56.101-192.168.56.105).

Before scanning the Windows XP server you would want to go to the terminal of the machine and type **ipconfig /all**, copy the IP (which should be along the lines of 192.168.56.10X).

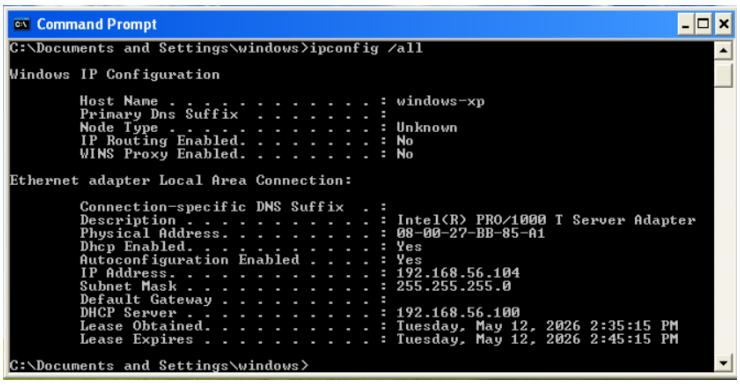
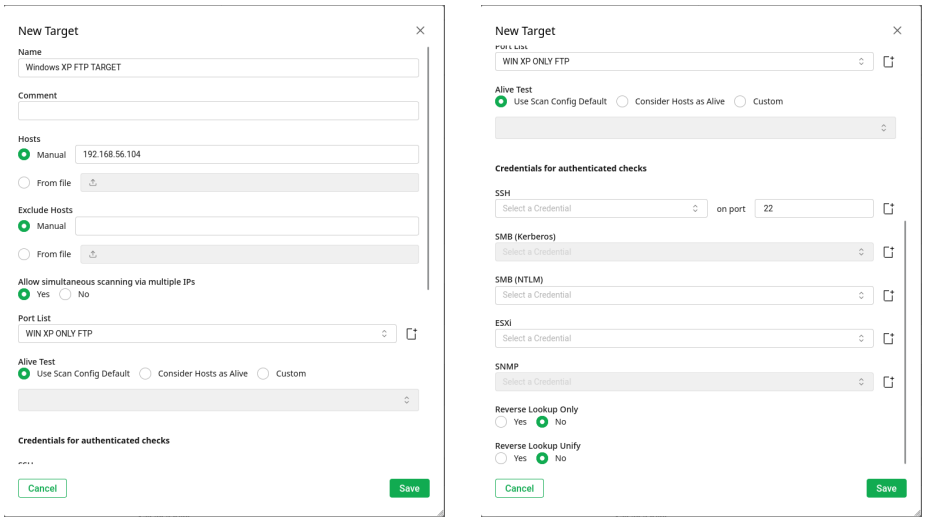


Figure 3.7: Terminal output of ipconfig command

After the IP was discovered, its time to configure the target like so:

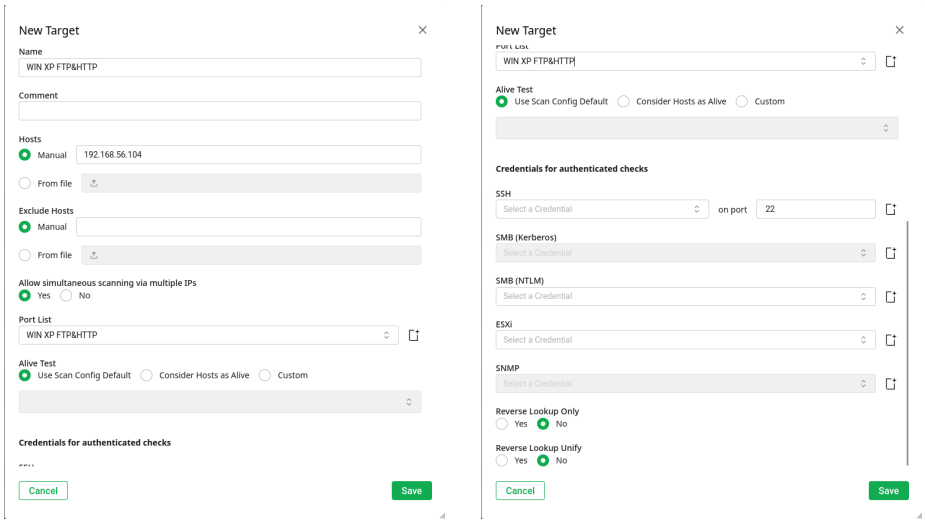


(a) FTP Target Step 1 (b) FTP Target Step 2

Figure 3.8: Configuration for the FTP only target

In this popup you will set target name, the IP (or range of IPs), the excluded hosts in case you have a range of IPs and some host are not to be scanned, the port list preset made before and the credentials presets that are set in the credentials window (not used in this example).

Now configure the targets also for the FTP and HTTP target like so:



(a) FTP & HTTP Target Step 1 (b) FTP & HTTP Target Step 2

Figure 3.9: Configuration for the FTP & HTTP target

Scan configs

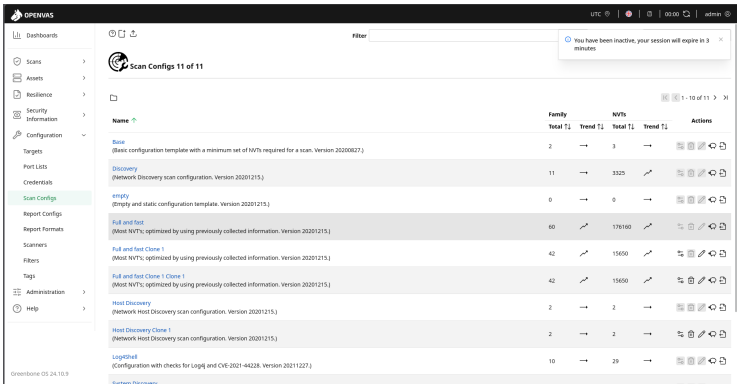


Figure 3.10: Scan configs window

The scan configuration is a configuration used during the scan, essentially it tells OpenVAS how to scan and what to scan. Essentially the Full and Fast preset looks at everything possible from Cisco appliances, Fedora security, Windows services etc. This is the reason why the database is so big, occupying tens of gigabytes. Since there are thousands of settings if we look at every possible configuration, what we will do is to configure a pre-set at a high level tayloring it for Windows XP.

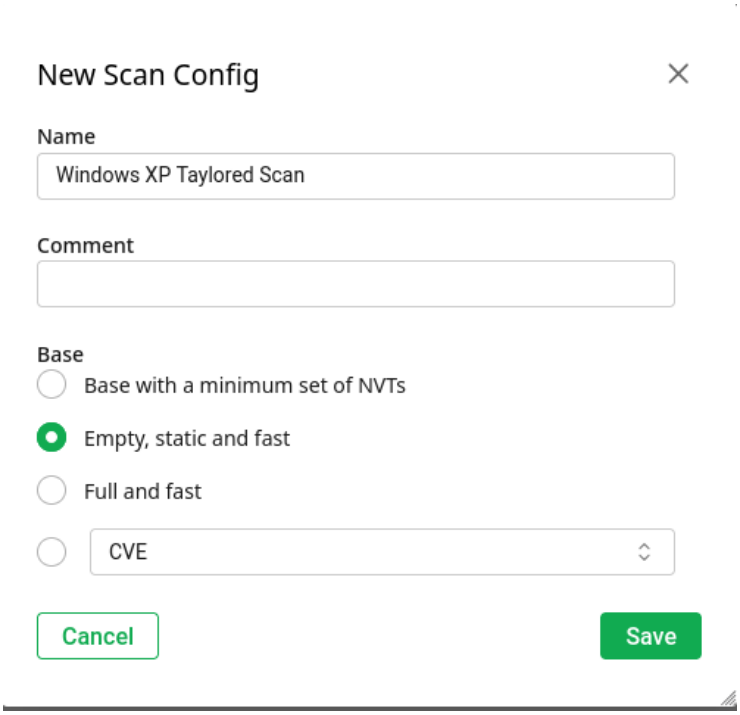
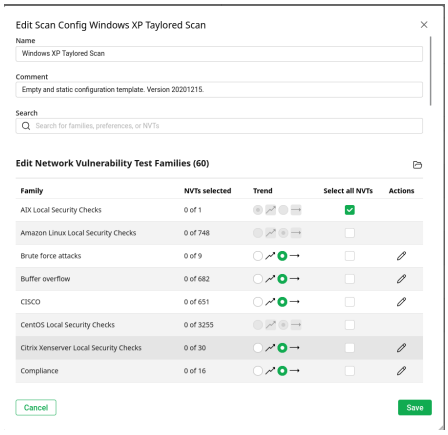
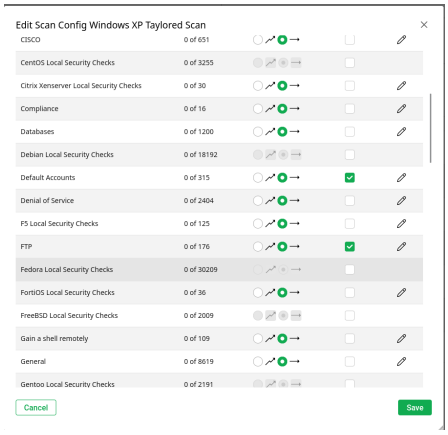


Figure 3.11: Taylored scan creation

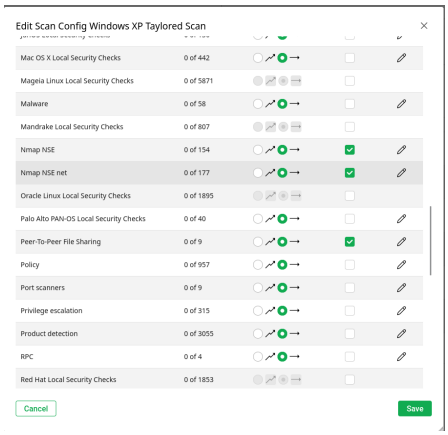
To create it select the Empty config, save it and then edit it by clicking to the pencil icon, select the following:



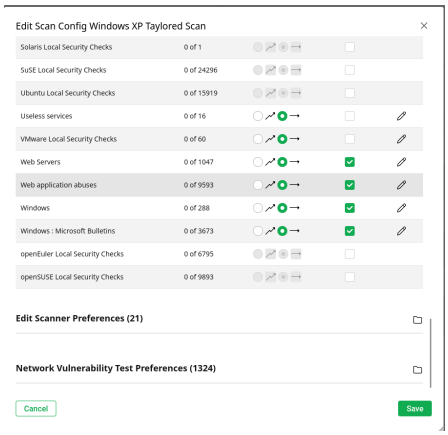
(a) FTP & HTTP Taylored scan options 1



(b) FTP & HTTP Taylored scan options 2



(a) FTP & HTTP Taylored scan options 3



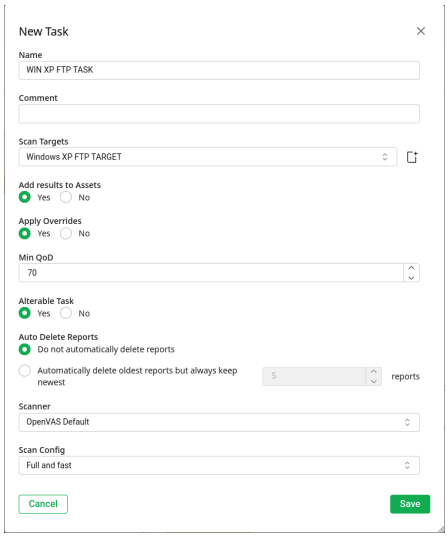
(b) FTP & HTTP Taylored scan options 4

Figure 3.13: Configuration for the FTP & HTTP taylored scan

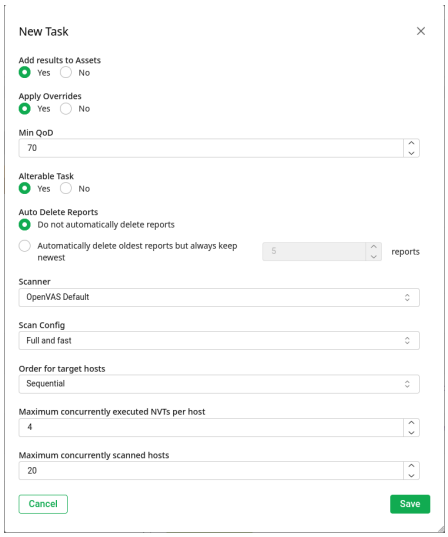
This is far from being a completely optimized scan for our purpose, since we are able to better tailor each section seen here by pressing on the pencil mark, but that would become time consuming since you would select/deselect hundreds of different things it scans, so for semplicity it is ignored in this experiment.

3.1.3 Starting the scan

Now we are ready to begin the scan, to do this go back to the tasks windows, click on the task add button and add all the settings created so far like this:



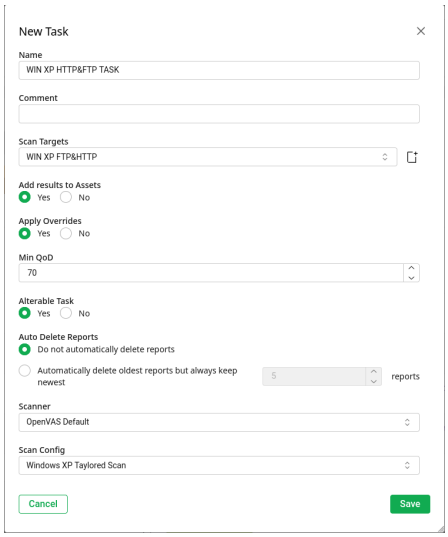
(a) FTP task 1



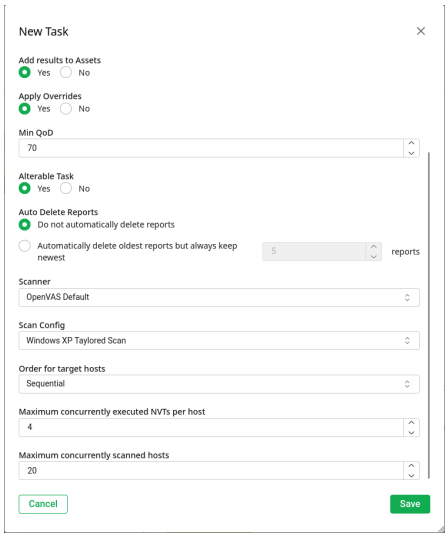
(b) FTP task 2

This task will scan Windows XP for the FTP service. Now you should have a new task created with the name you gave it, in this case **WIN XP FTP TASK** like so:

Now press the play button to queue it so that it will start, it will take approximately 10 minutes to complete.



(a) FTP & HTTP Taylored task 1



(b) FTP & HTTP Taylored task 2

Chapter 4

AI Usage Declaration

This report was prepared with the assistance of AI tools for drafting and editing support. The structure and the core contents of the report were designed by the author. All final content was reviewed and approved by the author.